FACULDADE DE TECNOLOGIA JOSÉ CRESPO GONZALES
ANALISE E DESENVOLVIMENTO DE SISTEMAS

MARIA JULIA LOUREIRO DA COSTA ALVES
RAFAELA SILVA RIGANTI
SOPHIA MARCELINO DE ARAUJO

DESENVOLVIMENTO E DOCUMENTAÇÃO DO PROJETO: IOTRACK

Sorocaba - SP
2026

MARIA JULIA LOUREIRO DA COSTA ALVES
RAFAELA SILVA RIGANTI
SOPHIA MARCELINO DE ARAUJO

DESENVOLVIMENTO E DOCUMENTAÇÃO DO PROJETO: IOTRACK

Projeto de desenvolvimento do IOTRACK, apresentado como critério avaliativo pelo professor ministrador da disciplina de Programação Multiplataforma.

Orientador: André Cassulino.

Sorocaba - SP

2026

SUMÁRIO

1 INTRODUÇÃO 4

2 PROBLEMA RESOLVIDO 4

3 ARQUITETURA 4

4 STACK MING 5

4.1 MQTT - Protocolo de Mensageria (Mosquitto) 5

4.2 Node-Read 6

4.3 InfluxDB 6

4.4 Grafana 7

5 CAMADA DE APLICAÇÃO 7

6 EXECUÇÃO DO PROJETO 8

7 DASHBOARDS 9

REFERÊNCIAS 14

LISTA DE FIGURAS

Figura 1 - Diagrama de Arquitetura 5
Figura 2 - Diagrama de Fluxo 5
Figura 3 - Pipeline Configuardo 6
Figura 4 - WOKIWI 9
Figura 5 - NodeRed 9
Figura 6 - InfluxDB 10
Figura 7 - Grafana 10
Figura 8 - FrontEnd 11
Figura 9 - AWS Portas 11
Figura 10 - AWS Instancia 12
Figura 11 - AWS DockerContainer 12
Figura 12 - DockerBackEnd 13

LISTA DE TABELAS

Tabela 1 - Dados Monitorados 4

1 INTRODUÇÃO

O IOTRack é uma solução completa de monitoramento de infraestrutura para racks de servidores, simulando um cenário real de data center. O sistema coleta continuamente dados de temperatura, corrente elétrica e status da porta do rack, processa essas informações em um pipeline IoT e as transforma em indicadores úteis para tomada de decisão operacional.

2 PROBLEMA RESOLVIDO

Racks de servidores são componentes críticos de infraestrutura. Falhas por superaquecimento, sobrecarga elétrica ou acesso indevido podem causar perda de dados e indisponibilidade de sistemas. O IOTRack permite que operadores monitorem o ambiente em tempo real e recebam alertas automáticos quando parâmetros ultrapassam limites seguros.

Dados Monitorados:

Tabela 1 - Dados Monitorados

Campo	Descrição	Faixa Simulada
Temperatura	Temperatura interna do rack (°C)	20.0 – 35.0 °C
Corrente Ampares	Consumo elétrico em Amperes	10.0 – 25.0 A
Porta Aberta	Status da porta do rack	true / false
Rack_Id	Identificador do rack	RACK-01
Timestamp	Horário da leitura (ISO 8601)	—

Fonte: Autoria própria

Regra de Negócio Implementada:

O backend calcula automaticamente a média de temperatura a cada 5 minutos e classifica o rack como:

- Estável: temperatura média ≤ 28°C
- Crítico: temperatura média > 28°C

3 ARQUITETURA

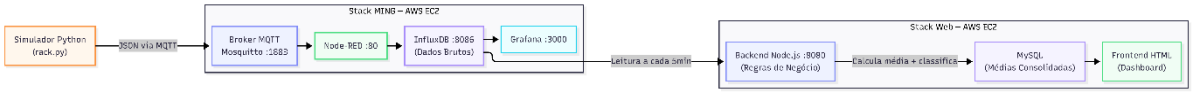
O IOTRack foi construído em duas etapas evolutivas que se complementam. A primeira estabelece o pipeline de dados em tempo real com a Stack MING. A segunda adiciona inteligência de negócio com uma API REST, banco relacional e interface web formando um sistema end-to-end completo, do sensor ao dashboard.

Toda a infraestrutura roda em containers Docker orquestrados pelo Docker Compose dentro de uma instância AWS EC2 (Ubuntu 22.04), com cada

serviço isolado em seu próprio container e comunicando-se pela rede interna do Compose.

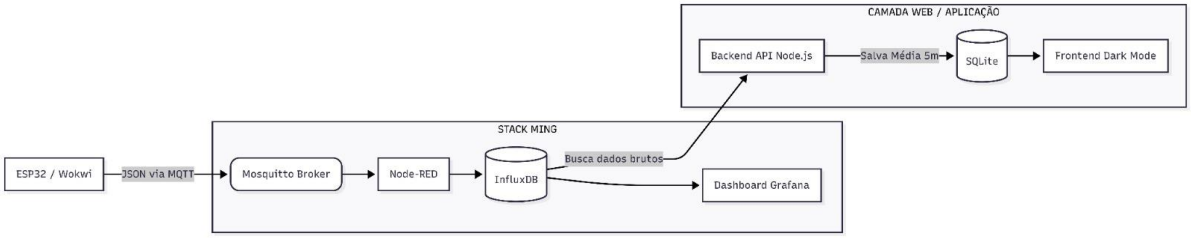
O projeto é dividido em duas etapas evolutivas que se integram:

Figura 1 - Diagrama de Arquitetura



Fonte: Autoria Própria

Figura 2 - Diagrama de Fluxo



Fonte: Autoria Própria

4 STACK MING

A Stack MING é o coração do pipeline IoT, responsável pela coleta, roteamento, armazenamento e visualização dos dados brutos em tempo real. O nome é um acrônimo das quatro tecnologias que a compõem: MQTT, InfluxDB, Node-RED e Grafana. Juntas, elas formam um fluxo contínuo onde cada dado gerado pelo simulador percorre toda a cadeia em menos de um segundo, chegando ao dashboard do Grafana praticamente em tempo real.

Todos os quatro serviços são definidos no docker-compose.yml e sobem juntos com um único comando, sem necessidade de instalação manual de dependências .

4.1 MQTT - Protocolo de Mensageria (Mosquitto)

Diferente do HTTP, o MQTT funciona em um modelo de Publicação/Assinatura. Isso reduz o overhead de rede e o consumo de bateria em dispositivos reais. O broker Mosquitto gerencia as filas de mensagens, garantindo que o dado chegue aos assinantes mesmo com oscilações de conexão.

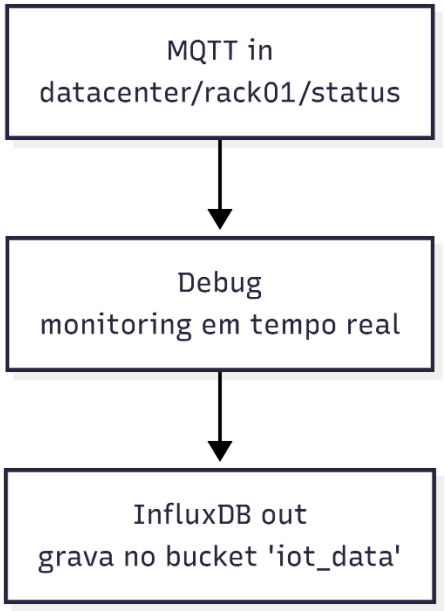
O MQTT (Message Queuing Telemetry Transport) é um protocolo leve de publicação/assinatura, ideal para dispositivos IoT com baixa largura de banda.

- Broker utilizado: Eclipse Mosquitto
- Porta: 1883
- Tópico: datacenter/rack01/status
- Papel: Recebe as mensagens JSON do simulador e as distribui para os assinantes (Node-RED)

4.2 Node-Read

O Node-RED é uma ferramenta de programação visual baseada em fluxos, desenvolvida pela IBM e hoje mantida pela OpenJS Foundation. No IOTRack, ele atua como o "encanador" do sistema: assina o tópico MQTT, recebe cada payload JSON publicado pelo simulador e o encaminha automaticamente para o InfluxDB tudo sem escrever uma única linha de código de integração.

Figura 3 - Pipeline Configuardo



Fonte: Autoria Própria

- Assina o tópico MQTT e recebe os JSONs
- Parseia automaticamente o payload
- Persiste cada leitura no InfluxDB com o measurement rack01

4.3 InfluxDB

O InfluxDB é um banco de dados especialmente projetado para armazenar séries temporais ou seja, dados que mudam continuamente ao longo do tempo. Diferente de bancos relacionais tradicionais, o InfluxDB indexa tudo

pelo timestamp e oferece uma linguagem de query própria (Flux) otimizada para agregar, filtrar e calcular médias em janelas de tempo. Isso o torna a escolha natural para dados de sensores IoT com alta frequência de leitura.

O InfluxDB recebe os dados do Node-RED a cada 5 segundos e os guarda com precisão de milissegundos. Ele também é consultado pelo backend via Flux Query para calcular a média de temperatura dos últimos 5 minutos.

4.4 Grafana

O Grafana é a interface de observabilidade do pipeline MING. Ele se conecta diretamente ao InfluxDB como fonte de dados e renderiza os valores em painéis interativos que atualizam automaticamente.

Como está configurado no projeto:

- Roda na porta 3000
- Fonte de dados: InfluxDB via Flux query
- Painéis criados para o IOTRack:
- Gráfico de linha: Temperatura ao longo do tempo.
- Métrica em tempo real: Corrente atual em Amperes.
- Indicador de status: Porta do rack aberta/fechada.

O Grafana consome os dados brutos do InfluxDB e é voltado para monitoramento contínuo em tempo real diferente do frontend web, que exibe as médias já consolidadas pelo backend.

5 CAMADA DE APLICAÇÃO

A Stack Web é a segunda etapa do projeto. Enquanto a Stack MING cuida do fluxo bruto de dados em tempo real, a camada de aplicação adiciona inteligência de negócio: ela consome os dados do InfluxDB, aplica uma regra de transformação, persiste o resultado processado e expõe tudo via API REST para o frontend.

É composta por três partes:

- Banco relacional: InfluxDB (Time-series) E SQLite (Relacional).
- Infraestrutura/Cloud: AWS EC2 (Ubuntu), Docker, Docker Compose.
- IoT & Integração: MQTT, Node-RED.

- Bancos de Dados: InfluxDB (Time-series) e SQLite (Relacional).

6 EXECUÇÃO DO PROJETO

Pré-requisitos

- Docker e Docker Compose instalados na instância EC2
- Python 3 com pip disponível (para o simulador pode rodar no Google Colab)
- Instância AWS EC2 com Ubuntu 22.04 e portas liberadas no Security Group
- Git para clonar o repositório

Passos para subir a Infraestrutura (Backend):

- Acesse o servidor ou máquina local contendo o Docker.
- Navegue até a pasta raiz ou pasta /backend.
- Execute o comando de orquestração do Compose:

  sudo docker-compose up -d –build

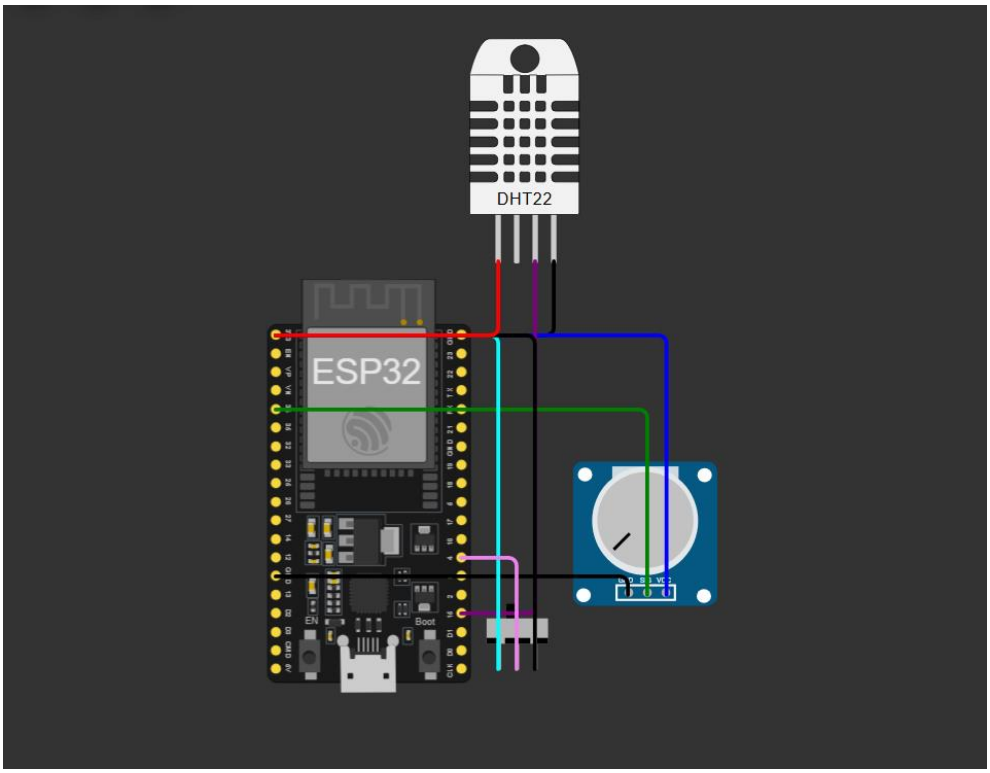
A API estará disponível recebendo conexões na porta 8080.

Passos para visualização (Frontend):

- Abra o arquivo index.html da pasta /frontend em qualquer navegador.
- Nota: É necessário garantir que a constante IP_AWS dentro da tag <script> do HTML esteja apontando para o endereço IP público atual da máquina hospedeira.
- A visualização estará disponível recebendo conexões na porta 8000.

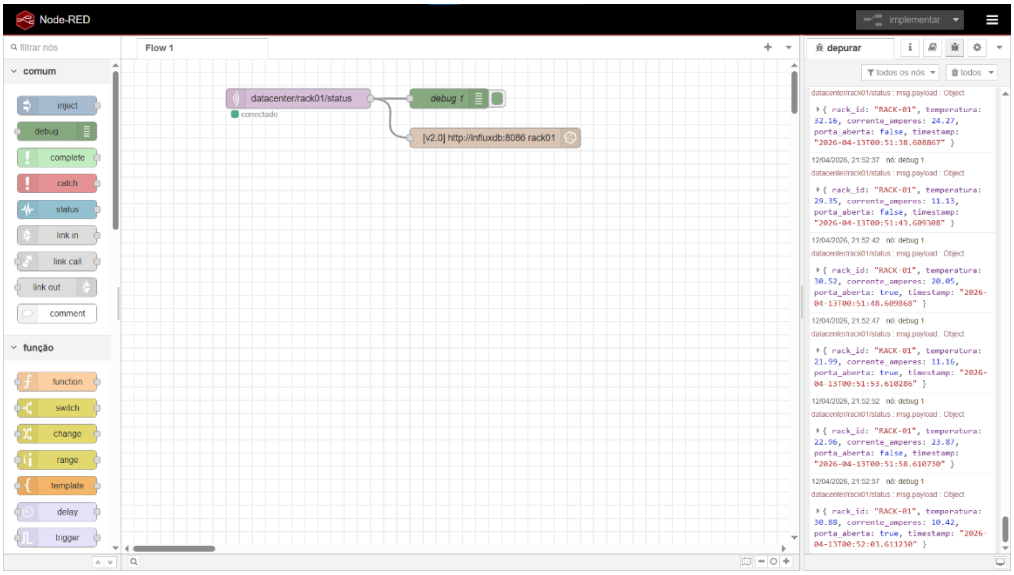
7 DASHBOARDS

Figura 4 - WOKIWI



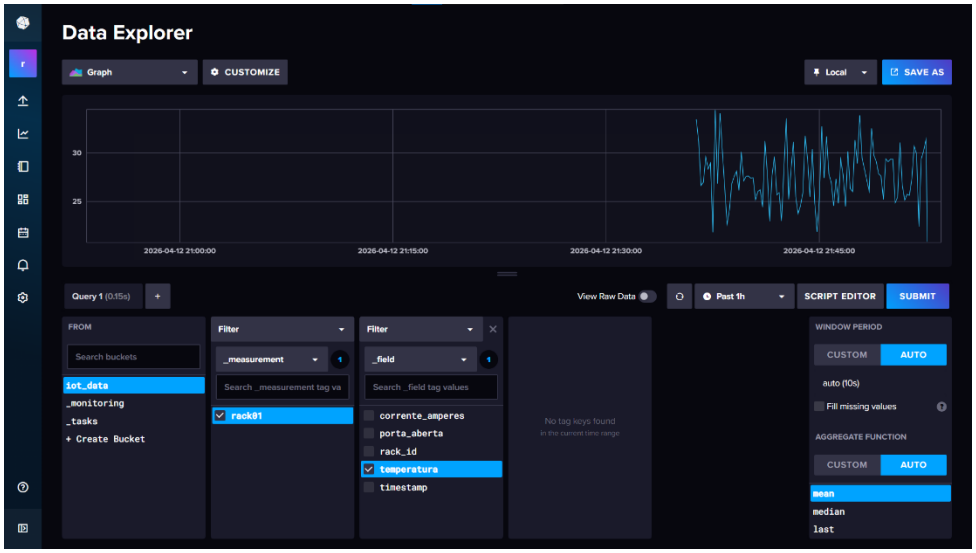
Fonte: Autoria Própria

Figura 5 - NodeRed



Fonte: Autoria Própria

Figura 6 - InfluxDB



Fonte: Autoria Própria

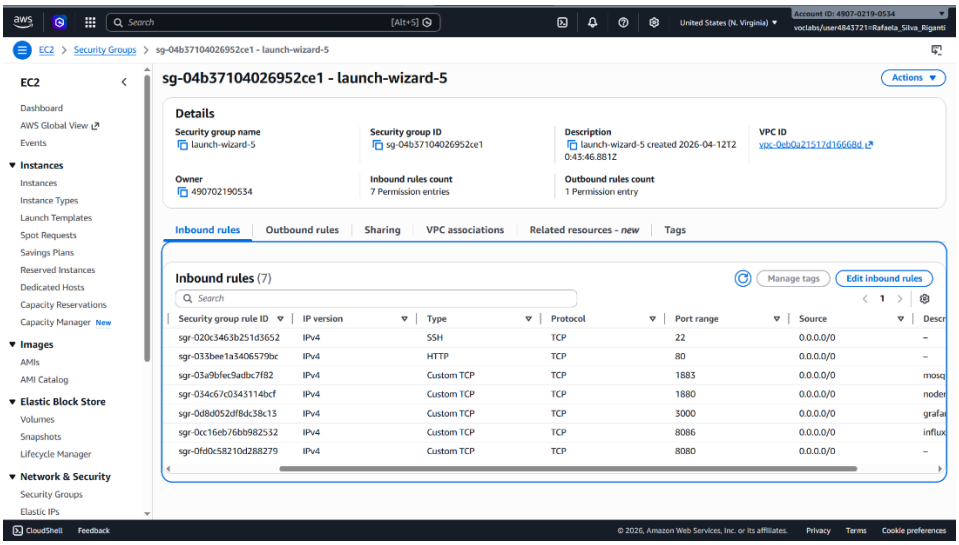
Figura 7 - Grafana

Fonte: Autoria Própria

Figura 8 - FrontEnd

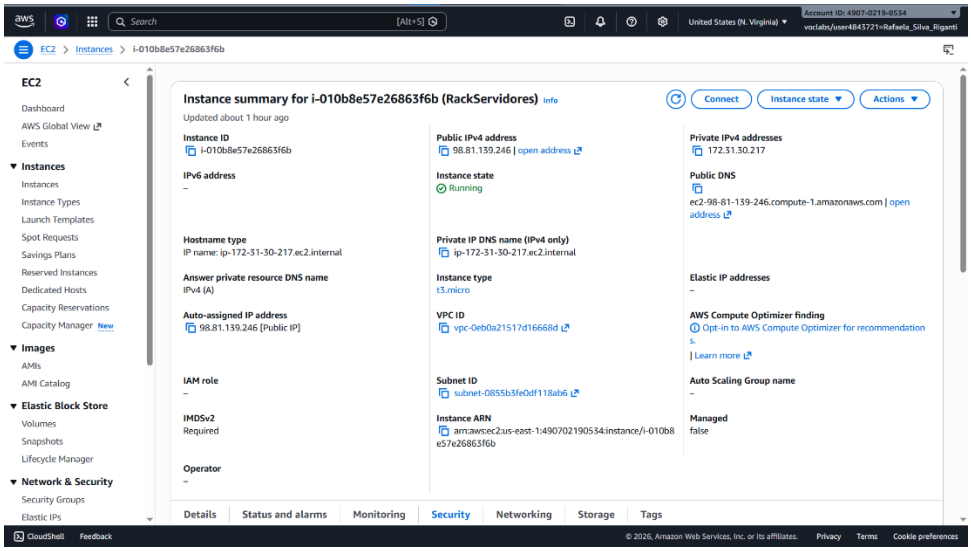
Fonte: Autoria Própria

Figura 9 - AWS Portas



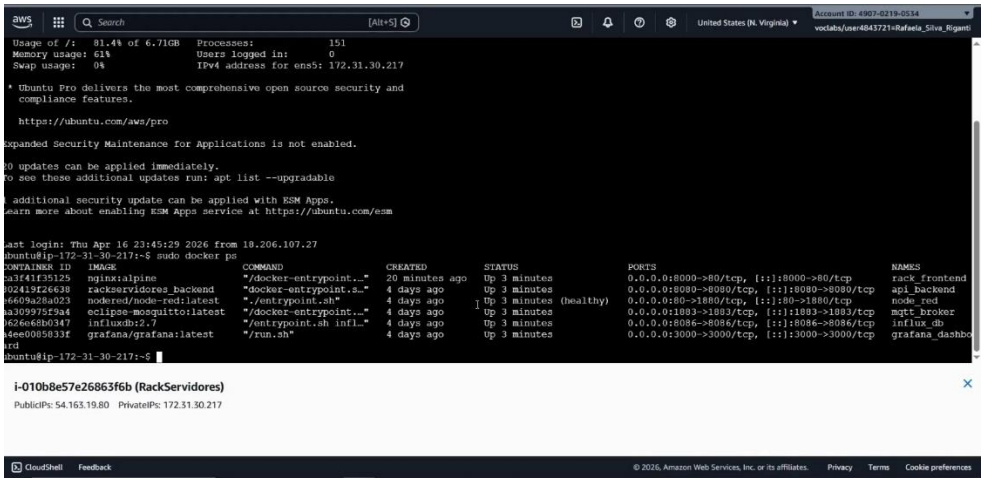
Fonte: Autoria Própria

Figura 10 - AWS Instancia



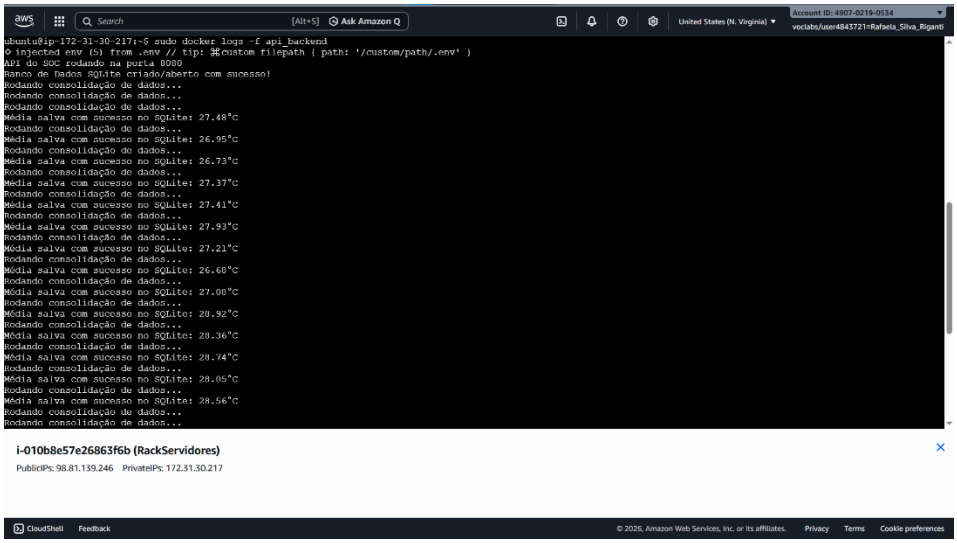
Fonte: Autoria Própria

Figura 11 - AWS DockerContainer



Fonte: Autoria Própria

Figura 12 - DockerBackEnd



Fonte: Autoria Própria

REFERÊNCIAS

Painel de controle. Disponível em: <https://awsacademy.instructure.com/>. Acesso em: 17 abr.. 2026.

rafaelariganti/IOTRack. Disponível em: <https://github.com/rafaelariganti/IOTRack>. Acesso em: 17 abr.. 2026.